RemediAI Copilot - user guide

Audience: operators, accountable owners, reviewers and local administrators.
Application: http://127.0.0.1:8082
Scope: the Docker-deployed `remedi-ai-copilot` variant.
Baseline: deployed commit `bec87e8`, 26 October 2026.

New opt-in feature: Real Kubernetes pod-event examples and live remediation progress. With that integration enabled, the four `k8s_*` examples operate on actual Docker Desktop Kubernetes resources and require owner approval for every remediation. The eight original scenarios described below remain synthetic. The baseline above predates this extension.

Repository-backed approval: GitHub remediation workflow changes the real Kubernetes owner action to **Approve & create PR**. It requires a reviewed repository diff, manual human merge and successful merge-commit CI before deployment. Use the application's `OWNER_TOKEN`, not a GitHub access token. Legacy live-only proposals cannot bypass this gate and require fresh analysis.

Contents

1. What the application does
2. Before you begin
3. Connect and navigate
4. Example: automatic low-risk recovery
5. Example: owner-approved worker recovery
6. Example: reject a recommendation
7. Example: dynamic AI diagnosis and review
8. Example: rollback and safe escalation
9. Scenarios and status reference
10. Audit, runbooks, observability and learning
11. PowerShell API examples
12. Local administration and deployment
13. Troubleshooting
14. Suggested demonstration and evidence checklist

1. What the application does

RemediAI demonstrates a governed incident-remediation workflow:

```
Synthetic incident -> evidence-based AI triage -> deterministic policy
  -> automatic low-risk action OR accountable owner approval
  -> simulated execution -> independent verification
  -> close or rollback -> audit and runbook-review candidate
```

Google ADK orchestrates read-only evidence tools and bounded model calls. The configured GitHub Copilot model recommends an action. Application policy, not the model, determines whether execution is allowed.

Important boundaries

- All incident signals, infrastructure actions and health probes are synthetic. Creating an incident does not restart a real worker or clear a real cache.
- Copilot inference is real when the provider is `copilot`; each incident can make multiple calls and consume your subscription allowance.
- An owner approval authorizes only the exact stored proposal for that incident.
- Dynamic cases receive a fresh reviewer invocation using the same configured model. This is not independent expert validation.
- Learning records candidates for human runbook review. It does not train the model or automatically deploy new runbooks.
- This is a single-identity pilot with application role tokens, not enterprise SSO, individually attributed human identity or a multi-tenant service.

2. Before you begin

2.1 Confirm the application is ready

Docker Desktop's Linux engine must be running. In PowerShell:

```
Set-Location "C:\Users\Kura Dheeraj\OneDrive\SNS_IT\SNS_IT_update\remedi-ai-copilot"
docker compose ps
Invoke-RestMethod -Uri "http://127.0.0.1:8082/readyz" -TimeoutSec 30
```

Expected readiness response:

```
{"status":"ready","provider":"copilot"}
```

The backend and frontend should both be `running` and `healthy`. Readiness proves provider authentication and configured-model discovery, not completed inference.

The deployment baseline uses `claude-sonnet-5`. Always check the model displayed by your current application; configuration can change.

2.2 Obtain the correct application role token

Open the private project `.env` locally in your editor, or obtain the appropriate application token from the authorized local administrator.

Credential	Purpose	Where used
`OPERATOR_TOKEN`	Submit synthetic incidents and inspect evidence	UI **Access token** or API bearer authentication
`OWNER_TOKEN`	Review, approve or reject a pending proposal	UI **Access token** or API bearer authentication

Credential	Purpose	Where used
`COPILOT_GITHUB_TOKEN`	Authenticate server-side Copilot inference	Backend configuration only; **never paste into the UI**
GitHub repository/runner credentials	Git and CI/CD operations	GitHub tooling/runner only; not application login

Never paste credentials into chat, documentation, screenshots, Git commits or support tickets. Do not display the entire configuration in terminal output. The `.env` file is ignored by Git but remains plaintext on disk.

Use separate roles deliberately. Possessing both tokens does not constitute independent human approval in a production governance process.

3. Connect and navigate

1. Open http://127.0.0.1:8082.
2. In **Access token**, enter the application operator token.
3. Select **Connect**.
4. Confirm the **operator session** badge appears.
5. Confirm **LLM connection** shows `READY: Configured provider ready`.
6. Check the provider/model badge shows the intended live provider.

The token stays in browser memory; it is not stored in browser local/session storage. A page reload requires reconnecting. **Disconnect** clears the active application connection. Protect your clipboard and screen.

Navigation item	Use
Operations	Submit scenarios, filter the queue, inspect recommendations and approve/reject
Runbooks	Inspect the three reviewed simulation runbooks and rollback descriptions
Observability	Inspect reported model usage, inference latency and outcomes
Learning	Review feedback candidates from completed execution outcomes

The queue list refreshes periodically, approximately every six seconds. Selecting an incident opens its authenticated live event stream. Expand **Evidence & telemetry** on an event to inspect its payload.

Switching roles: enter the other application token and select **Connect**. The selected incident is cleared on connection changes; select the same incident again from the queue. Copy its ID before changing roles if needed.

4. Example: automatic low-risk recovery

Goal: demonstrate safe automatic remediation inside the approved boundary.

1. Connect as **operator** and open **Operations**.
2. Under **Inject a synthetic incident**, select: **Cache pressure - Low risk / automatic recovery**.
3. Select **+ Simulate incident** once.
4. Record the new incident ID in **Live execution trace**.

5. Wait for the triage and policy decision. Do not repeatedly resubmit while it runs.

Expected successful behavior:

- Incident title: **Checkout cache pressure**.
- The model inspects trusted synthetic metrics and retrieves the cache runbook.
- The recommendation is `clear_cache`, shown as **clear cache**.
- Cited evidence includes `EV-METRICS-01` and `RB-CACHE-01`.
- Adequate confidence and valid policy permit automatic execution.
- The simulator saves a checkpoint and applies the action.
- Three independent simulated health samples pass.
- Status becomes **closed** and the trace includes: **Three independent simulated probes passed; incident closed.**

Select **Export audit JSON** after the trace is complete to retain evidence.

Not a guaranteed live-model outcome: if confidence is insufficient, the incident may await owner approval. Invalid output, unavailable service or unsupported evidence must fail or escalate, not be treated as recovery. Do not approve a questionable proposal just to make a demonstration pass.

5. Example: owner-approved worker recovery

Goal: demonstrate the high-impact approval gate.

Operator

1. Connect as operator.
2. Select **Worker stalled - High impact / owner approval**.
3. Select **+ Simulate incident**.
4. Select **Payments worker stalled** and wait for **awaiting approval**.
5. Review the proposed **restart worker** action and cited worker runbook.
6. Confirm **Accountable owner approval required** is displayed.

The operator cannot approve: approval controls are only available to the owner. No simulated restart should occur before approval.

Accountable owner

1. Enter the owner application token and select **Connect**.
2. Confirm **owner session**.
3. Reselect the incident by its ID/title.
4. Review the recommendation, evidence, risk and rollback plan in **Runbooks**.
5. Enter an **Approval reason**, for example:

> Reviewed the stalled-worker evidence, affected scope and checkpoint rollback. Approve the exact proposed simulated restart.

6. Select **Approve change**.
7. Wait for execution and verification.
8. Confirm **closed** only after three successful recovery samples.

Reasons must contain **8 to 500 characters after trimming**. Approval is valid only while the incident is `awaiting_approval`; repeated or stale approval requests are rejected. The owner role cannot submit a new incident.

6. Example: reject a recommendation

1. As operator, create another **Worker stalled** incident.
2. Wait until it is **awaiting approval**.
3. Connect as owner and reselect that incident.
4. Enter a reason such as:

> Reject this proposed restart; the operating window has not been approved.

5. Select **Reject**.

Expected outcome: **rejected**, with an owner approval-decision audit event and no execute event. Rejection is terminal; there is no UI control to reopen or approve the rejected incident. A new submission creates a separate incident.

7. Example: dynamic AI diagnosis and review

Dynamic scenarios require causal diagnosis, not only a metric threshold. They run triage and then a fresh ADK resolution-review invocation. These extra calls consume subscription allowance.

UI scenario	Synthetic evidence	Intended remedy / gate
Dynamic schema mismatch	Cached schema v1 conflicts with valid origin v2; cache utilization is below the standard pressure threshold	Clear cache; low-risk auto path only if confidence and independent review pass
Dynamic worker deadlock	Healthy cache despite high utilization; a worker has stopped heartbeating after deadlock	Restart worker; owner approval required
Dynamic demand saturation	Sustained increased demand, healthy workers and high CPU; restarting would reduce capacity	Scale workers; owner approval required

Walkthrough: dynamic demand saturation

1. Connect as operator.
2. Select **Dynamic demand saturation - LLM capacity recommendation / owner approval**.
3. Submit once and select the new incident.
4. Expand **Evidence & telemetry** to inspect observed diagnostic context and runbook citations.
5. Confirm the recommendation addresses capacity, rather than restarting healthy workers.
6. Inspect the `judge` event for the reviewer decision.
7. Connect as owner and reselect the incident.
8. Review the proposed `scale_workers` action, capacity risk and rollback plan.
9. Approve only if appropriate, providing a reason:

   > Reviewed sustained demand, healthy worker heartbeats, temporary capacity impact and checkpoint restoration. Approve the exact simulated scale-out.

10. Confirm execution, three health samples and the terminal outcome.

Reviewer disagreement or low confidence requires owner judgment. A reviewer error fails closed. Matching recommendations do not override a high-risk owner gate. If the remedy does not address the evidence, reject it.

8. Example: rollback and safe escalation

8.1 Failed recovery restores a checkpoint

1. Connect as operator.
2. Select **Recovery failure - Verify / rollback**.
3. Submit and inspect the trace.
4. If policy permits execution, observe the cache action followed by failed probes.
5. Confirm **rolled back**, not **closed**.
6. Expand the rollback event and inspect checkpoint-restoration evidence, including `rollback_verified`.

The simulator intentionally does not recover for this scenario. Successful rollback restores its checkpoint and records the outcome for owner investigation. This is application-level simulated rollback, not Docker image rollback.

8.2 Unknown or untrusted signals do not authorize a write

- **Unknown signal - Uncertain / escalate:** unsupported trusted signatures should escalate for investigation, without execution.
- **Untrusted signal - Injection / policy boundary:** injected instructions in synthetic log text are data, not authority. Expected safe outcome is escalation without shell access, credential access or self-approval.

Invalid live model output can instead produce **failed**. In either case, do not describe the incident as resolved or assume a repair occurred.

9. Scenarios and status reference

Scenario identifiers for API clients

UI name	API `scenario`	Intended successful/safe outcome
Cache pressure	`cache_pressure`	`closed`, or owner gate if confidence is low
Worker stalled	`worker_stall`	Owner decision, then `closed` if approved and verified
Unknown signal	`unknown_signal`	`escalated`, no write
Recovery failure	`verification_failure`	`rolled_back` after failed recovery checks
Untrusted signal	`prompt_injection`	`escalated`, no write
Dynamic schema mismatch	`dynamic_cache`	Reviewed low-risk recovery, possibly owner-gated
Dynamic worker deadlock	`dynamic_worker`	Reviewed proposal plus owner decision
Dynamic demand saturation	`dynamic_capacity`	Reviewed capacity proposal plus owner decision

These are curated scenarios, not general free-text monitoring ingestion.

Incident statuses

UI labels replace underscores with spaces.

API status	Meaning / next action
`queued`	Accepted and waiting for the single worker
`triaging`	Evidence gathering, model diagnosis and applicable review are running
`ready`	Eligible for execution; typically transient
`awaiting_approval`	Owner must review and approve or reject
`approved`	Owner approved; waiting for execution
`executing`	Allowlisted simulated action is in progress
`verifying`	Independent synthetic health checks are running
`closed`	Three recovery samples passed
`rolled_back`	Recovery failed; simulated checkpoint restoration succeeded
`escalated`	No permitted remedy; investigate without execution
`rejected`	Owner declined; no action executes

API status	Meaning / next action
`failed`	Workflow error or interrupted work; operator investigation required

Terminal statuses are `closed`, `rolled_back`, `escalated`, `rejected` and `failed`. An event stream marked **complete** means it ended, not necessarily that recovery succeeded. Check the incident's terminal status and verification evidence.

10. Audit, runbooks, observability and learning

Audit export

1. Open **Operations** and select the incident.
2. Wait for stream completion where possible.
3. Expand relevant **Evidence & telemetry** details.
4. Select **Export audit JSON**.

The browser downloads `remedi-audit-<incident-id>.json`, containing `incident` and `events`. Export reflects events loaded in that browser at export time; exporting mid-workflow may be incomplete. For a fresh server-side event list, use the API example below.

Review `triage`, `reason`, `adk`, `observe`, `decision`, applicable `judge` and `approval`, then `execute`, `verify`, `close` or `rollback`, and `learn`. Not every stage appears in every path. Review the actual outcome, not a fixed checklist of event names.

Audit messages contain concise operational justifications, not hidden chain-of-thought. Treat exported incident/evidence data as controlled material; review before sharing even though current scenarios are synthetic.

Runbooks

Open **Runbooks** to inspect:

Runbook	Action	Risk
`RB-CACHE-01`	`clear_cache`	Low
`RB-WORKER-01`	`restart_worker`	High
`RB-CAPACITY-01`	`scale_workers`	High

Each card shows a description and rollback plan. There is no UI runbook editor or arbitrary shell-command execution.

Observability

Open **Observability** for **Inference & governance telemetry**:

- LLM calls and provider-reported input/output tokens.
- Calls with unavailable token usage.

- Outcome distribution and per-call p95 inference latency.
- Prompt version, maximum agent steps and auto-confidence threshold.

The aggregate API scope is the **most recent 100 incidents**. The queue fetches the default most recent 50; older items may not appear there. The filter operates on the loaded list by service, status or title, not a full-history server search.

Missing token usage is unknown, not zero. Copilot subscription usage is not a dollar invoice and is not unlimited. p95 is per successful recorded inference step, not end-to-end incident duration. Failed requests may still consume allowance even if successful-call usage totals do not include them.

Authenticated Prometheus metrics are available at `/api/metrics`; process counters reset when the backend restarts. Persistent incident/audit records do not.

Learning

1. Open **Learning**.
2. Select **Refresh feedback** to fetch newly completed execution outcomes.
3. Review the outcome and runbook-review candidate alongside the incident audit.

Feedback does not apply itself. Review, test and version any proposed runbook change through the normal development/deployment process. Not every terminal incident produces a feedback candidate; execution outcomes such as closure and verified rollback do.

11. PowerShell API examples

Use the same application role tokens as the UI, not GitHub or Copilot credentials. Examples below target the local Docker instance. Do not record a terminal transcript containing secrets. These examples work in Windows PowerShell 5.1.

11.1 Read a role token without placing it in command history

```
$base = "http://127.0.0.1:8082"
$secureOperator = Read-Host "Enter the application OPERATOR_TOKEN" -AsSecureString
$operatorToken = [System.Net.NetworkCredential]::new("", $secureOperator).Password
$operatorHeaders = @{ Authorization = "Bearer $operatorToken" }

Invoke-RestMethod -Uri "$base/api/config" -Headers $operatorHeaders
Invoke-RestMethod -Uri "$base/api/provider-status" -Headers $operatorHeaders
```

The plain token exists in process memory to form the request header. Do not print `$operatorToken`, `$operatorHeaders` or session variables. Clear variables when finished; clearing them does not guarantee immediate memory erasure.

11.2 Submit and follow one incident

This invokes the live configured model and can consume allowance.

```
$key = "user-guide-" + [Guid]::NewGuid().ToString("N")
$requestHeaders = @{
    Authorization = "Bearer $operatorToken"
    "Idempotency-Key" = $key
}
$body = @{ scenario = "cache_pressure" } | ConvertTo-Json
$created = Invoke-RestMethod -Method Post -Uri "$base/api/incidents" `
    -Headers $requestHeaders -ContentType "application/json" -Body $body
$id = $created.incident.id
$created | Select-Object created, @{ Name = "IncidentId"; Expression = { $_.incident.id } }

$stopStates = @("closed", "rolled_back", "escalated", "rejected", "failed", "awaiting_approval")
$deadline = (Get-Date).AddMinutes(6)
do {
    $incident = Invoke-RestMethod -Uri "$base/api/incidents/$id" -Headers $operatorHeaders
    $incident | Select-Object id, status, provider, model
    if ($stopStates -contains $incident.status) { break }
    if ((Get-Date) -ge $deadline) {
        throw "Polling timed out. Inspect incident $id; do not blindly resubmit."
    }
    Start-Sleep -Seconds 3
} while ($true)
```

A successful POST returns HTTP **202** with an `incident` object and `created` boolean. This means accepted, not resolved. Repeating the same scenario with the same idempotency key retrieves the original incident without creating another. Reusing that key with a different scenario returns **409**.

Use a new key for a deliberately new incident. If a client times out after POST, reuse the original key/body to check whether it was accepted before retrying.

11.3 Retrieve current audit events and save a review copy

```
$incident = Invoke-RestMethod -Uri "$base/api/incidents/$id" -Headers $operatorHeaders
$events = @(Invoke-RestMethod -Uri "$base/api/incidents/$id/events" -Headers $operatorHeaders)
$audit = @{ incident = $incident; events = $events }
$output = Join-Path $env:TEMP ("remedi-audit-" + $id + ".json")
$audit | ConvertTo-Json -Depth 30 | Set-Content -LiteralPath $output -Encoding UTF8
Write-Host "Audit saved to $output"
```

Save snapshots only to an approved location and review their contents before sharing. This is a client-side export; there is no separate `/audit` API endpoint.

11.4 Review a pending incident as owner

Do not run approval automatically for arbitrary incidents. First inspect the specific plan, evidence, risk and rollback.

```
$secureOwner = Read-Host "Enter the application OWNER_TOKEN" -AsSecureString
$ownerToken = [System.Net.NetworkCredential]::new("", $secureOwner).Password
$ownerHeaders = @{ Authorization = "Bearer $ownerToken" }
```

```
$pending = Invoke-RestMethod -Uri "$base/api/incidents/$id" -Headers $ownerHeaders
$pending | Select-Object id, status, plan

if ($pending.status -ne "awaiting_approval") {
    throw "Incident is not awaiting approval; no decision was submitted."
}

# Deliberately choose approve or reject after reviewing this exact proposal.
$decision = Read-Host "Enter approve or reject"
if ($decision -notin @("approve", "reject")) { throw "Invalid decision." }
$reason = Read-Host "Enter your review reason (8-500 characters)"
$approvalBody = @{ decision = $decision; reason = $reason } | ConvertTo-Json
Invoke-RestMethod -Method Post -Uri "$base/api/incidents/$id/approval" `
    -Headers $ownerHeaders -ContentType "application/json" -Body $approvalBody |
    Select-Object id, status
```

After approval, poll again as in section 11.2 to obtain the execution outcome. Approval itself does not establish recovery. Operator approval requests return **403**; repeated decisions after the pending state return **409**.

11.5 Clean up local credential variables

```
Remove-Variable operatorToken, operatorHeaders, requestHeaders, secureOperator, `
    ownerToken, ownerHeaders, secureOwner -ErrorAction SilentlyContinue
```

Here `SilentlyContinue` applies only to removing optional variables that may not have been created; do not suppress API, authentication or deployment failures.

Endpoint reference

Endpoint	Method	Access
`/readyz`	GET	Public readiness; provider check
`/healthz`	GET	Docker frontend nginx liveness; not backend/provider readiness
`/api/config`	GET	Operator or owner
`/api/provider-status`	GET	Operator or owner
`/api/incidents`	GET	Operator or owner; optional `limit` 1-100
`/api/incidents`	POST	Operator; `Idempotency-Key` header required, 8-100 characters
`/api/incidents/{id}`	GET	Operator or owner
`/api/incidents/{id}/events`	GET	Operator or owner; optional nonnegative `after` sequence
`/api/incidents/{id}/stream`	GET	Operator or owner; authenticated SSE
`/api/incidents/{id}/approval`	POST	Owner; pending exact proposal only

Endpoint	Method	Access
`/api/runbooks`, `/api/feedback`, `/api/telemetry`	GET	Operator or owner
`/api/metrics`	GET	Operator or owner; Prometheus text, not JSON
`/api/docs`	GET	API documentation UI

12. Local administration and deployment

Existing Docker installation

From the project folder, `docker compose start` starts existing stopped containers without rebuilding or replacing their image versions:

```
docker compose start
docker compose ps
Invoke-RestMethod -Uri "http://127.0.0.1:8082/readyz" -TimeoutSec 30
```

If containers do not exist, `start` does not create a deployment. Follow CI/CD setup or the README for initial provisioning. Starting existing containers does not load configuration edits into them.

To stop only this application without deleting data:

```
docker compose stop
```

Do not run global Docker cleanup or `down -v`: deleting the named volume destroys incident history and runtime data. Docker Desktop and the local machine must remain running while the application is in use.

Configuration and credentials

Docker uses a separate supported server-side Copilot credential; native Windows CLI login does not authenticate the container. Keep `PROVIDER=copilot`, `COPILOT_ALLOW_EGRESS=true`, and a model available to the authenticated account. Do not confuse GitHub repository access with Copilot entitlement.

After deliberately editing the private configuration, recreate the backend through a successful deployment. A simple container restart does not reload its environment. The CI/CD workflow reads the private file in place, using the repository variable `REMEDI_LOCAL_ENV_FILE` as a path. Never store the token in that variable or pass it as a workflow input.

For supported account-token creation, rotation and policy requirements, use GitHub's authentication documentation and local authentication guidance.

Run the deployment workflow

1. Open the repository's **Actions** tab.

2. Select **CI and local Docker deployment**.
3. Select **Run workflow**, branch `main`.
4. Leave `live_smoke` disabled for readiness/contract-only deployment, or enable it when you authorize a synthetic live incident and associated model usage.
5. Select **Run workflow** and inspect both jobs.

The workflow validates code, audits dependencies, builds images, and runs fixture browser tests on hosted Linux before local deployment. Pull requests never deploy to your local machine. The Windows runner must be online and able to access Docker and the private configuration.

Live smoke requires a closed low-risk incident and three recovery samples. Failed verification restores prior images when available, preserves the data volume, and keeps the workflow failed. It does not roll back the database, erase audit entries or refund model usage.

Avoid manually rebuilding untagged images during deployment. Use the pipeline to retain tested commit-to-image traceability.

Native alternative

The optional native installation runs at http://127.0.0.1:8081, with separate host-runtime authentication and state paths. Follow the README for setup, login, probe and start commands. The original GPU sibling normally uses port 8080; it is not the Docker Copilot instance on 8082.

13. Troubleshooting

Symptom	Check and safe next step
Page does not open	Confirm Docker Desktop is running, `docker compose ps` shows the frontend running, and use port **8082**
`401` / cannot connect	Use the application operator/owner token, not a GitHub token. Reconnect after a page reload
`403` submitting an incident	The owner cannot ingest; connect as operator
`403` approving	The operator cannot approve; connect as owner
`409`	Stale/repeated approval, idempotency-key conflict or queue capacity; inspect the error and current incident before retrying
`422`	Invalid scenario/body/header/reason; use exact scenario IDs and a reason of 8-500 trimmed characters
`429`	Rate limit; honor `Retry-After` and stop repeated submissions
`503` / provider BLOCKED	Check server credential, Copilot entitlement, allowed model and egress; use **Recheck provider** after correction
Token valid for GitHub but inference unavailable	Repository API access does not prove Copilot/model permission. An administrator must verify supported SDK authentication and model discovery
Ready but incident `failed`	Readiness is not inference. Inspect that incident's error/audit; runtime, timeout, quota and response-validation errors fail closed

Symptom	Check and safe next step
Low-risk case awaits approval	Confidence or reviewer judgment can trigger a gate. Review rather than bypassing it
Approval panel disappears after role switch	Connection changes clear selection. Reselect the pending incident
Trace is `disconnected`	Check backend readiness, then reselect the incident or reconnect; inspect persisted events before creating duplicates
Trace `complete` but incident not closed	Stream completion also occurs for failed, escalated, rejected and rolled-back incidents
Learning list looks stale	Select **Refresh feedback**
Incident missing from queue	Clear the filter; list is limited to recent records. Retrieve a known ID through the API
Missing usage / no USD cost	Expected when SDK usage is absent; Copilot totals are not an invoice
New workflow queued	Check Actions for CI/concurrency and whether the registered local runner is online
Dependency audit fails	Do not suppress the advisory or force mismatched packages. Follow the maintained ADK compatibility-build process in CICD

For support, record the incident ID, status, provider/model, time, sanitized error category and Actions run URL if relevant. Do not share `.env`, container environment dumps, raw SDK dumps, tokens or browser authorization codes. Administrators should sanitize logs locally before sharing them.

14. Suggested demonstration and evidence checklist

Run scenarios deliberately; each live incident can make several billable calls. Do not run all examples merely to verify documentation.

Step	Demonstrate	Evidence to retain
1	Connect as operator; check provider ready	Provider/model and readiness, no tokens
2	Cache pressure	Recommendation, citations, closed status, three probes
3	Worker stalled; approve as owner	Pending gate, reason, owner decision, verified closure
4	Another worker incident; reject	Rejected status and no execution
5	Dynamic schema/capacity case	Causal evidence, reviewer event, correct risk gate
6	Recovery failure	Failed probes, restored checkpoint and rolled-back status
7	Unknown/untrusted signal	Escalation without execution
8	Inspect observability and learning	Reported usage, outcome distribution, review candidates

The verified baseline deployment completed a live low-risk cache incident with three recovery samples. This does not certify every live example, model or production scenario. Outcomes depend on valid model responses and current account/service availability.

Further reading

- Architecture, technical and workflow diagram viewer
- Diagram PDF
- Architecture and production-release scope
- ADK dynamic diagnosis and policy gates
- Authentication, billing and operations
- CI/CD and private ADK maintenance
- Successful baseline deployment run